

CSCI 49900: Project Proposal

Alex Haskel
alex.haskel98@login.cuny.edu

James Lacoste
james.lacoste56@login.cuny.edu

Kailun You
kailun.you08@login.cuny.edu

I. PROJECT OVERVIEW

The Language Model Lab project is an interactive platform meant to demonstrate the mechanics of Large Language Model (LLM) inference optimization and model behavior through internal interventions without the need for special tools or programming.

II. MOTIVATION

Large language models are increasingly ubiquitous in many aspects of industry, academia, and research. Yet, the high level of complexity of such models means that the mechanics of model inference are difficult for users to understand, observe directly, or experiment with. Currently, exploring the role of specific internal components, as well as the impact of optimization methods on model responses and performance, often requires specialized knowledge, tools, or programming experience.

Our project will provide a platform for researchers, computer scientists, and hobbyists to directly explore questions about LLM behavior, optimization, architecture, and performance.

III. GOALS

The central part of our platform will be an interactive display in which users can prompt a model, perform various modifications and optimizations on it, and compare the responses resulting from their experimental configuration with the response produced by a baseline configuration using default settings. The user should also be able to compare model responses and inference performance through various metrics.

On the side of model behavior, the ultimate goal is to allow for as much interactive experimentation as possible, while providing users clear explanations of what their modifications are actually doing to the model. On the optimization side, the user will be able to choose among several quantization schemes, and observe how these affect performance and output as well.

Some examples of questions a user could interactively explore using our platform would be:

- What is quantization, how does it work, and how does it affect LLM behavior and performance?
- How do decoding parameters such as temperature, top-k, top-p, etc., work, and how do they interact with internal changes to the model's activations?
- Are specific kinds of learned information stored in specific model layers?

- How do attention mechanisms affect model outputs? Are specific aspects of model output affected by specific attention heads?
- How robust are a model's learned behaviors to noise and distortion?
- Do distortions in specific internal components consistently produce types of distortions in outputs?

Some tentative experiments that could be added later include:

- System prompt experiments.
- For advanced users: Granular control over up projection, down projection, or gate projection matrix weights.
- Activation steering experiments.

IV. PROJECT SCOPE

The initial scope of the project is to develop a functional web platform that allows users to interact with a small language model, modify selected inference parameters and internal model components, and compare experimental outputs against a baseline. The platform will also include selected optimization techniques, such as quantization and custom GPU kernels, together with performance metrics. More advanced interventions, visualizations, and explanatory material will be added as time permits. Training a language model from scratch or supporting large numbers of different model architectures is outside the intended scope of the project.

V. EXPECTED OUTCOME

The expected outcome is a working web application that allows users to submit prompts, configure model and inference settings, and compare baseline and experimental model behavior side by side. The application should demonstrate several kinds of internal intervention and inference optimization while presenting relevant performance information in an accessible way. The final platform is intended to serve primarily as an interactive educational and experimental tool, rather than as a production-level AI assistant or chatbot.

VI. EVALUATION METHODS

We will evaluate the project through functional testing of the application and through quantitative comparisons. Backend and API components will be tested for correct handling of requests, invalid inputs, and model interventions. Optimization methods such as quantization and custom kernels will be evaluated using metrics such as inference latency, token throughput, and memory usage relative to a baseline implementation. Internal interventions will be tested for reproducibility and for

their ability to produce noticeable changes in model outputs or probability calculations. The complete frontend-to-backend workflow will also be tested to ensure that users can reliably configure experiments and view the resulting outputs and metrics.

VII. TECHNICAL APPROACH

For model optimization, we will write the quantization and kernel functions using CUDA C++, an extension of standard C++ developed by NVIDIA that allows developers to write code for GPU devices.

Most aspects of model testing, loading and inference will be written in Python, as PyTorch is the main library that will be used for these.

Given our hardware constraints, the compute demand imposed by the models, and our particular interests, we agreed that models in the range of 500M to 2B parameters would work best. We plan to use models in the Qwen3 family, since these strike a reasonable balance in terms of complexity, size, and response quality.

Our backend will be written in Python. This is convenient due to the need to extensively interact with Python libraries like PyTorch and Transformers. Our backend API will be implemented using Pydantic for schema validation, FastAPI for our API framework, and Uvicorn as our web server.

Our front end will be written using React, possibly with some visualizations using D3.js.

VIII. DIVISION OF RESPONSIBILITIES

There are three main aspects that we have identified in this project:

- **Model optimization:** When it comes to optimization, we are interested in demonstrating a few techniques. One of these is quantization, i.e., the reduction of the numerical precision of certain parameters, in order to make a model faster and less memory-intensive. Another is the use of custom kernel components, GPU-executed functions designed to accelerate the computations involved in language model inference, with the aim of improving baseline implementations. James will be the primary teammate involved in implementing these, and coming up with ideas of how to explain and expose these to the user.
- **General application backend design and model interventions:** The actual online platform needs to have correct backend logic and a usable API that can take input from the frontend and return correctly formatted responses. Alex will be mainly in charge of implementing and testing these basic aspects of the backend. Related to this is the implementation of the experiments involving internal modifications of the model. These require writing hooks, which are custom functions that execute automatically during a model's forward or backward passes. They allow for the inspection or modification of intermediate activations, inputs, and gradients without changing the core model architecture. Alex will also be

in charge of coming up with interesting experiments, determining if they are viable, and writing hooks and backend logic for them.

- **Frontend and User Experience:** The last part of our platform is the user-facing component. These include well-designed user controls that can be used to modify various model parameters, graphical explanations, and the general web design of the platform. Kailun will be the main teammate in charge of this aspect of the project.

IX. SEMESTER TIMELINE

We intend for our timeline to remain fluid, but here are some landmarks that we expect to reach in the near future:

- 1) A minimal viable product should allow a user to prompt a model, set some very simple experimental parameters (such as decoding parameters), and view the responses of their experimental and baseline runs side by side, with some simple metrics displayed. We expect to reach this around the middle of the semester.
- 2) Once the above is achieved, the next step would be to integrate into this MVP more complex experimental controls, such as internal ablations, and more detailed performance metrics. We will also have to properly test that the platform is able to handle multiple simultaneous requests.
- 3) During the second half of the semester, once a minimal version is tested and working properly, we will perform additional research to determine which other interventions and experiments may be reasonable to add. Some of these are listed in the "Goals" section. We may also add pages with further explanations about model optimization, internal architecture, visualizations, and detailed explanations for experimental parameters.